LLM Inference Gateway: SLA-Aware Admission Control and Performance Evaluation

PART- II

Alis

https://softmaxxer101.github.io/
https://x.com/import_alis

Date: 1st November 2026

Abstract- This technical abstract outlines the SLA-aware admission-control mechanism for an LLM inference gateway. The gateway estimates request completion time using recent service-time observations and rejects requests predicted to miss their deadline. Experiments across multiple SLA budgets measured acceptance rate, latency, compliance, and rejection reasons. Compared with capacity-based admission and a baseline gateway, the SLA-aware gateway reduced latency while maintaining full observed SLA compliance for accepted requests.

Index Terms- LLM inference, SLA-aware admission control, request deadlines, latency prediction, queue management, vLLM, inference gateway.

I. SLA-AWARE ADMISSION CONTROL

In the previous part we have successfully created a basic inference gateway engine. In this blog, we aim to make a SLA- aware admission policy on our current existing gateway system. I have uploaded the updated gateway and newly created load_test_sla.py file – click here

Experiment 1 – Run the first SLA experiment at 3000ms budget

Start the gateway :

```
# MAX_ACTIVE=1 MAX_QUEUE=4 QUEUE_DEADLINE=15 UPSTREAM_TIMEOUT=30 LOG_PATH=results/phase2_baseline.jsonl uvicorn gateway:app --host 127.0.0.1 --port 9000
```

```
# python load_test_sla.py 10
```

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/production_vllm$ python load_test_sla.py 10
SLA budget: 3000 ms
Sent 10 simultaneous requests -> status counts: {429: 9, 200: 1}
200s: median 0.59s, p95 0.59s, max 0.59s
Successful responses within client-observed budget: 1/1
429 rejection reasons: {'unspecified': 9}
Slowest 429 rejection: 0.08s
```

```
INFO:     Uvicorn running on http://127.0.0.1:9000 (Press CTRL+C to quit)
INFO:     127.0.0.1:55250 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:55262 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:55274 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:55288 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:55300 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:55314 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:55330 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:55346 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:55354 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:55242 - "POST /v1/chat/completions HTTP/1.1" 200 OK
```

The above experiment shows that even though maximum 5 requests can be accepted in the gateway. Only 1 could enter the gateway, the rest got rejected becoz the it probably exceeded the sla's 3000ms budget. To verify this we whether the requests got rejected we will run another experiment to check whether the requests got rejected because of queue_deadline or sla_budget.

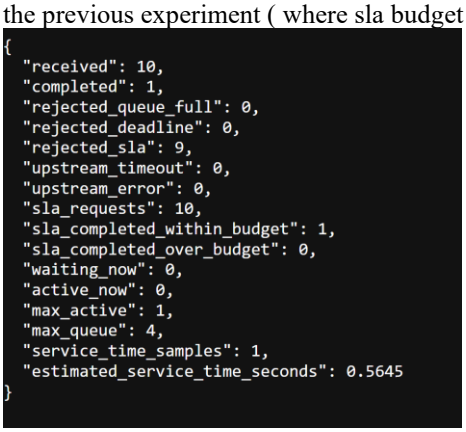
Run :

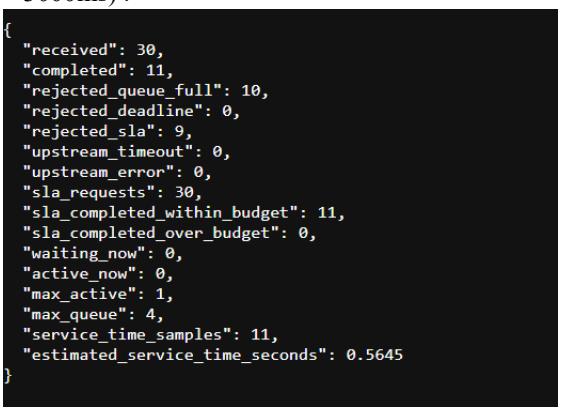
SLA_BUDGET_MS=10000 python load_test_sla.py 10
The above command shows that we have increased the sla's budget from 3000ms to 10000ms.

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/production_vllm$ SLA_BUDGET_MS=10000 python load_test_sla
.py 10
SLA budget: 10000 ms
Sent 10 simultaneous requests -> status counts: {429: 5, 200: 5}
200s: median 1.09s, p95 1.44s, max 1.77s
Successful responses within client-observed budget: 5/5
429 rejection reasons: {'unspecified': 5}
Slowest 429 rejection: 0.02s
```

```
INFO:     127.0.0.1:51354 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:51362 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:51364 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:51366 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:51294 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:51310 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO:     127.0.0.1:51320 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO:     127.0.0.1:51326 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO:     127.0.0.1:51340 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO:     127.0.0.1:51344 - "POST /v1/chat/completions HTTP/1.1" 200 OK
```

This clears up that the rejection in the previous experiment was due to sla's budget npt queue_deadline. To confirm this, we can also check the metrics to verify. The "rejected_sla" in figure 1 (left image) value confirms that 9/10 requests were rejected because of sla in the previous experiment (where sla budget = 3000ms) .





In the second figure (sla budget = 10000ms), 5 request were accepted and rest were rejected as the sla budget was increased.

Comparision : SLA budget 3000ms vs 10000ms

Measurement	**3-second budget**	**10-second budget**
Requests sent	10	10
Successful (200)	1	5
Rejected (429)	9	5
Median successful latency	0.59 s	1.09 s
P95 successful latency	0.59 s	1.44 s
Slowest rejection	0.08 s	0.02 s

Let's compare the gateway with and without SLA header using 10 requests.

Run for baseline test:
#python load_test.py 10

Run for SLA test:
#SLA_BUDGET_MS=10000 python load_test_sla.py 10

Compaision table:

Metric	Baseline	SLA-aware (10 seconds)
Requests sent	10	10
Successful (200)	5	5
Rejected (429)	5	5
Median successful latency	1.11 s	0.96 s
P95 successful latency	1.95 s	1.26 s
Maximum successful latency	1.95 s	1.64 s
Slowest rejection	0.03 s	0.02 s

Both the gateway types, show mostly same results.

Experiment : 2 – Let's set a tighter budget of SLA budget = 1000ms

Run :
#SLA_BUDGET_MS=1000 python load_test_sla.py 10

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/production_vllm$ SLA_BUDGET_MS=1000 python load_test_sla.
py 10
SLA budget: 1000 ms
Sent 10 simultaneous requests -> status counts: {429: 8, 200: 2}
200s: median 0.71s, p95 0.57s, max 0.84s
Successful responses within client-observed budget: 2/2
429 rejection reasons: {'unspecified': 8}
Slowest 429 rejection: 0.04s
```

Result :

Metric	10-second budget	1-second budget
Requests sent	10	10
Successful (200)	5	2
Rejected (429)	5	8
Median successful latency	0.96 s	0.71 s
Maximum successful latency	1.64 s	0.84 s
Slowest rejection	0.02 s	0.04 s

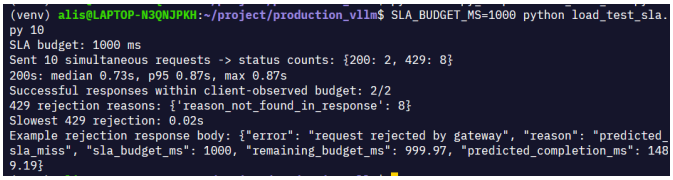
```
INFO:     127.0.0.1:50734 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:50750 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:50754 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:50768 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:50772 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:50782 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:50786 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:50790 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:50722 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO:     127.0.0.1:50728 - "POST /v1/chat/completions HTTP/1.1" 200 OK
```

Only 2/10 requests were accepted in 1000ms (allowed requests = 5, 2/5 filled) budget because of the tight SLA-budget, whereas for the 10000ms budget 5/5 requests were accepted (allowed requests = 5 , 5/5 filled).

Fix needed= P95 calculation, unspecified rejection reason.

Experiment 3: (Fixed P95 calculation and unspecified rejection reason) Updated file – click here

Run :
#SLA_BUDGET_MS=1000 python load_test_sla.py 10



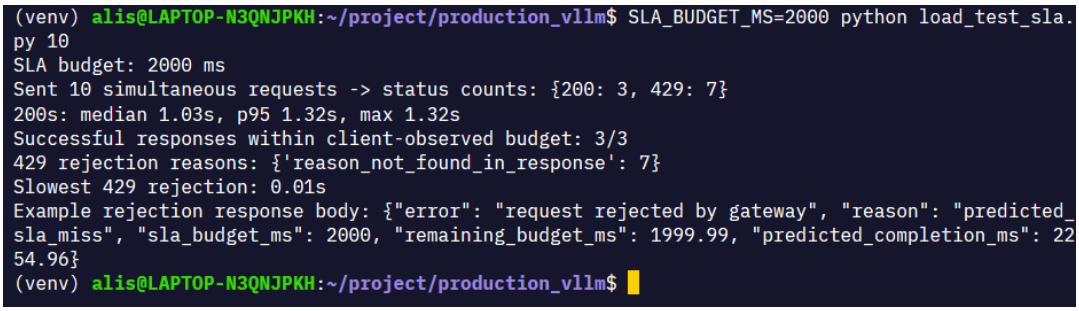
The rejection reason is now specified and reflects as below in a pretty-print:


```
{
  "error": "request rejected by gateway",
  "reason": "predicted_sla_miss",
  "sla_budget_ms": 1000,
  "remaining_budget_ms": 999.97,
  "predicted_completion_ms": 1489.19
}
```


The above shows that the other requests were rejected because their predicted calculation time was +1400ms which is +400 more than the alloted 1000ms budget. The other 2 requests which were accepted completed within the sla budget and hence were accepted.

Experiment 4: Test a 2 second budget

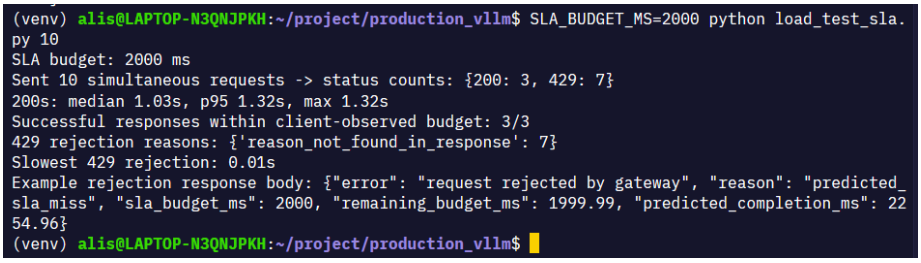
Run:
#SLA_BUDGET_MS=2000 python load_test_sla.py 10



The above results show that our hypthosis: a tighter sla-budget would reject mre requests is kinda starting to prove right.

Comparsion table: 10s vs 2s vs 1s

SLA budget	Successful (200)	Rejected (429)	Successful requests within budget
1 second	2	8	2/2
2 seconds	3	7	3/3
10 seconds	5	5	5/5



In the 2s budget, the alloted budget was 2000ms. The remianing budget was 1999.99ms.
The remaining 2000-elapsed time = 1999.99ms
The elapsed time= 0.01ms which was ultilised by the gateway to process the request is subtracted from the original budget, making the budget shrink to 1999.99ms budget. Hence, the remaining budget for all the remaining requests has become 1999.99ms instead of 2000ms (*). The predicted budget is the budget predicted for anyone of the 7 rejected requests which exceeds the sla budget provided and cannot be accepted.

Experiment 5: 5 second budget

Run:
#SLA_BUDGET_MS=5000 python load_test_sla.py 1

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/production_vllm$ SLA_BUDGET_MS=5000 python load_test_sla.
py 10
SLA budget: 5000 ms
Sent 10 simultaneous requests -> status counts: {200: 5, 429: 5}
200s: median 1.30s, p95 2.21s, max 2.21s
Successful responses within client-observed budget: 5/5
429 rejection reasons: {'reason_not_found_in_response': 5}
Slowest 429 rejection: 0.06s
Example rejection response body: {"error": "request rejected by gateway", "reason": "queue_full
"}
(venv) alis@LAPTOP-N3QNJPKH:~/project/production_vllm$
```

Results (in pretty-print):

Metric	**Result**
Total requests	10
Successful requests (200)	5
Rejected requests (429)	5
Median successful latency	1.30 s
p95 successful latency	2.21 s
Successful requests within the 5-second budget	5/5
Slowest rejection	0.06 s

5/10 requests were accepted, making the other 5 remaining requests rejected. The rejection reason this time was not because of the sla budget, instead it was because the queue limit was filled and 5/5 queue slots were
5/10 requests were accepted, making the other 5 remaining requests rejected. The rejection reason this time was not because of the sla budget, instead it was because the queue limit was filled and 5/5 queue slots were equiped. Hence, we can conclude that as we increase the sla budget to 5 seconds, the rejection reason changes and the rejections are not hardcoded by default to sla rejection.

Experiment : 6 – Revise the load_test_sla.py (fix the rejection reason specification in the output)

The updated file should replace this section :

```
#reason = ""

if isinstance(body, dict):
    detail = body.get("detail", "")
    reason = body.get("reason", "")

    if isinstance(detail, dict):
        reason = detail.get("reason", reason)
    elif isinstance(detail, str):
        reason = detail
```

with:

```
#reason = ""

if isinstance(body, dict):
    reason = body.get("reason", "")

    if not reason:
        detail = body.get("detail", "")

        if isinstance(detail, dict):
```

```
            reason = detail.get("reason", "")
        elif isinstance(detail, str):
            reason = detail
```

After making the changes, recompile your current load_test_sla.py file and run 10 load requests over 1s vs 2s vs 3s vs 4s vs 6s. Below in the table, I have summarized the output patterns across these varying sla-budget.

Comparision Table : 1s vs 2s vs 3s vs 4s vs 6s

SLA budget	Accepted (200)	Rejected (429)	Main rejection reason
1 second	2	8	Predicted SLA miss*
2 seconds	3	7	Predicted SLA miss
3 seconds	5	5	Predicted SLA miss
4 seconds	5	5	Queue full
6 seconds	5	5	Queue full

The above table shows the wide difference between rejection reason over different sla-budget. The 1s, 2s and 3s sla budget rejected due to predicted sla miss and others did because of queue slots being maxxed out.

Experiment: 7 - SLA-admission experiments

Run :

```
#cd ~/project/production_vllm
source venv/bin/activate
mkdir -p results
# Task 2.17: collect acceptance, SLA compliance, latency, and rejection reasons
python run_sla_experiments.py | tee results/task_2_17_output.txt
# Task 2.15 / 2.16: capacity comparison and repeatability
python compare_admission.py | tee results/task_2_15_2_16_output.txt
# Inspect generated result files
ls -lt results/*.json
# Create a compact summary for Tasks 2.17–2.19
python - <<'PY'
import json
from pathlib import Path
files = sorted(Path("results").glob("*.json"), key=lambda p: p.stat().st_mtime, reverse=True)
for path in files[:5]:
    try:
        data = json.loads(path.read_text())
    except Exception:
        continue
    print(f"\n===== {path} =====")
    if "capacity_based" in data and "sla_aware" in data:
        for policy in ("capacity_based", "sla_aware"):
            trials = data[policy]
            print(f"\n{policy.upper()}")
            for trial in trials:
                print(json.dumps(trial.get("summary", {}), indent=2))
    elif isinstance(data, list):
        for record in data:
            print(json.dumps({
                "budget_ms": record.get("sla_budget_ms"),
                "trial": record.get("trial"),
                "summary": record.get("summary"),
            }, indent=2))
PY
```

Summarized Table for Results:

SLA budget	Accepted / Total	Acceptance rate	SLA compliance	Avg. median latency	Avg. P95 latency	Main rejection reason
2 seconds	2.3 / 10	23.3%	100%	0.72 s	0.90 s	Predicted SLA miss
4 seconds	7 / 10	70.0%	100%	1.34 s	2.07 s	Predicted SLA miss
6 seconds	10 / 10	100%	100%	1.54 s	2.85 s	None
Admission policy	Accepted / Total	Acceptance rate	Rejections	Avg. median latency	Avg. P95 latency	SLA result
Capacity-based	30 / 30	100%	None	1.67 s	2.83 s	No SLA filtering
SLA-aware, 2-second budget	12 / 30	40%	18 predicted SLA misses	0.89 s	1.32 s	12/12 accepted requests met SLA

The above experiment shows that the SLA-aware admission accepts fewer requests but maintains SLA compliance and produces lower latency for accepted requests.

Experiment : 8 - Final Validation of SLA-Aware Admission Control

Run :

```
#cd ~/project/production_vllm
source venv/bin/activate

mkdir -p results

# Task 2.14: 10-second SLA with queue capacity 9
MAX_ACTIVE=1 MAX_QUEUE=9 QUEUE_DEADLINE=5 UPSTREAM_TIMEOUT=30 \
sed 's/BUDGETS_MS = \[2000, 4000, 6000\]/BUDGETS_MS = [10000]/' \
run_sla_experiments.py > run_sla_10s.py

MAX_ACTIVE=1 MAX_QUEUE=9 QUEUE_DEADLINE=5 UPSTREAM_TIMEOUT=30 \
python run_sla_10s.py | tee results/task_2_14_output.txt

# Task 2.18: baseline gateway comparison
python load_test.py 10 | tee results/task_2_18_baseline_output.txt

# SLA-aware comparison using a generous 10-second budget
SLA_BUDGET_MS=10000 python load_test_sla.py 10 \
  | tee results/task_2_18_sla_output.txt

# Task 2.19: collect all final experiment artifacts
echo "===== FINAL RESULT FILES =====" | tee results/task_2_19_summary.txt
ls -lt results/*.json results/*.txt | tee -a results/task_2_19_summary.txt

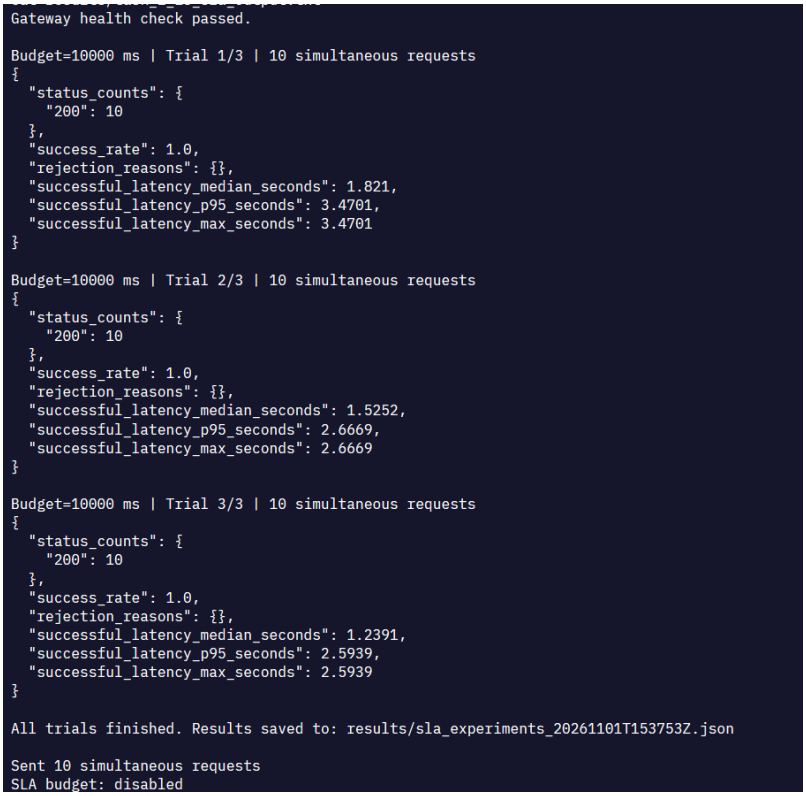
echo "===== SLA EXPERIMENT OUTPUT =====" | tee -a results/task_2_19_summary.txt
cat results/task_2_14_output.txt | tee -a results/task_2_19_summary.txt

echo "===== BASELINE OUTPUT =====" | tee -a results/task_2_19_summary.txt
cat results/task_2_18_baseline_output.txt | tee -a results/task_2_19_summary.txt

echo "===== SLA-AWARE OUTPUT =====" | tee -a results/task_2_19_summary.txt
cat results/task_2_18_sla_output.txt | tee -a results/task_2_19_summary.txt
```

Summarization of Output in Table:

Test configuration	Requests	Successful	Acceptance rate	Median latency	P95 latency	Rejections
10-second SLA, Trial 1	10	10	100%	1.82 s	3.47 s	0
10-second SLA, Trial 2	10	10	100%	1.53 s	2.67 s	0
10-second SLA, Trial 3	10	10	100%	1.24 s	2.59 s	0
Baseline gateway	10	10	100%	1.85 s	3.14 s	0
SLA-aware gateway	10	10	100%	1.18 s	2.28 s	0



Final verdict:

Experiment	Result
10-second SLA, 3 trials	30/30 successful; 100% acceptance; average median latency 1.53 s; average P95 latency 2.91 s; no rejections
Baseline gateway	10/10 successful; median latency 1.85 s; P95 latency 3.14 s
SLA-aware gateway, 10-second budget	10/10 successful; median latency 1.18 s; P95 latency 2.28 s

Phase 3 starts next blog onwards!